Line-Search Filter Differential Dynamic Programming for Optimal Control with Nonlinear Equality Constraints

Ming Xu[1], Stephen Gould[2] and Iman Shames[3]

***Abstract*— We present FilterDDP, a differential dynamic programming algorithm for solving discrete-time, optimal control problems (OCPs) with nonlinear equality constraints. Unlike prior methods based on merit functions or the augmented Lagrangian class of algorithms, FilterDDP uses a step filter in conjunction with a line search to handle equality constraints. We identify two important design choices for the step filter criteria which lead to robust numerical performance: 1) we use the Lagrangian instead of the cost in the step acceptance criterion and, 2) in the backward pass, we perturb the value function Hessian. Both choices are rigorously justified, for 2) in particular by a formal proof of local quadratic convergence. In addition to providing a primal-dual interior point extension for handling OCPs with both equality and inequality constraints, we validate FilterDDP on three contact implicit trajectory optimisation problems which arise in robotics.**

I. Introduction

Discrete-time optimal control problems (OCPs) are used in robotics for motion planning tasks such as minimum-time quadrotor flight [1], autonomous driving [2], locomotion for legged robots [3]–[5], obstacle avoidance [6]–[8] and manipulation [9]–[12]. The differential dynamic programming (DDP) class of algorithms [13] are of interest to the robotics community due to their efficiency, as well as other benefits for control, such as dynamic feasibility of iterates and a feedback policy provided as a byproduct of the algorithm [14], [15].

To date, extending DDP for nonlinear equality constraints is relatively underexplored. Equality constraints are required for OCPs involving inverse dynamics [15], [16], challenging contact implicit planning problems [10], [11], [17]–[20] and minimum time waypoint flight [1]. Most existing algorithms for handling equality constraints adopt an adaptive penalty approach based on the *augmented Lagrangian* (AL) [21]–[23]. However, AL methods yield limited formal convergence guarantees [24, Ch. 17], motivating a DDP algorithm based on a framework with stronger convergence properties, i.e., a line-search filter approach [25], [26], which is adopted in the popular nonlinear programming solver IPOPT [27].

As a result, we propose FilterDDP, a DDP line-search filter algorithm which complements existing works based on merit functions for handling equality constraints [15], [28]. We identify two important design choices when designing the filter algorithm which are essential for robust numerical performance: 1) we replace the cost with the Lagrangian as one of the two filter criterion and, 2) for the stopping criteria and backward pass Hessians, we replace the value function gradient with an estimated dual variable of the dynamics constraints. Both 1) and 2) are justified rigorously, with 2) via a proof of local quadratic convergence. We also validate 1) and 2) with an ablation study in the simulation results.

Another contribution of this paper relates to the aforementioned theoretical analysis of constrained DDP algorithms. First, we mathematically show that one of FilterDDP's sufficient decrease conditions is analogous to the Armijo condition [24, pg. 33] in unconstrained optimisation. Second, we provide a proof of local quadratic convergence of the iterates within a neighbourhood of a solution. This proof generalises the existing proof of local quadratic convergence of unconstrained DDP for scalar states and controls [29] to the the constrained, vector-valued setting. Our final contribution is a primal-dual interior point extension to FilterDDP for handling inequality constraints as well as equality constraints.

An implementation of FilterDDP is provided in the Julia programming language[1]. Our numerical experiments evaluate FilterDDP on 300 OCPs derived from three challenging classes of motion planning problems with nonlinear equality constraints: 1) a contact-implicit cart-pole swing-up task with Coulomb friction [19], 2) an acrobot swing-up task with joint limits enforced through a variational, contact-implicit formulation [19] and, 3) a contact-implicit non-prehensile pushing task [10]. Our experiments indicate that FilterDDP offers significant robustness gains and lower iteration count compared to the existing state-of-the-art equality-constrained DDP algorithms [15], [22], while being faster (and more amenable to embedded systems) compared to IPOPT [27].

A. Notation used in this paper

Let $[N] = \{1, 2, \ldots, N\}$. A function $p(x) : \mathcal{X} \to \mathcal{P}$ for some sets $\mathcal{X}$ and $\mathcal{P}$ is defined as being $O(\|x\|^d)$ for positive integer d if there exists a scalar C such that $\|p(x)\| \leq C\|x\|^d$ for all $x \in \mathcal{X}$. A vector of ones of appropriate size is denoted by e. Denote the collection of indexed vectors $\{y_k\}_{k=t}^{N}$ by $y_{t:N}$ and we use the convention $(x, u) = (x^\top, u^\top)^\top$. Denote the element-wise product by $\odot$. Finally, we use the following convention for derivatives: If f is a scalar valued function $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$, then $\nabla_u f(x, u) \in \mathbb{R}^{1\times m}$ and $\nabla^2_{xu} f(x, u) \in \mathbb{R}^{n\times m}$. If $g : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^c$ is a vector valued function, then $\nabla_x g(x, u) \in \mathbb{R}^{c\times n}$ and $\nabla^2_{xu} g(x, u) \in \mathbb{R}^{c\times n\times m}$.

[1]Ming Xu is with the School of Computer and Communication Sciences, EPFL (e-mail: mingda.xu@epfl.ch).

[2]Stephen Gould is with the School of Computing at the Australian National University (e-mail: stephen.gould@anu.edu.au).

[3]Iman Shames is with the Department of Electrical and Electronic Engineering, the University of Melbourne (email: iman.shames@unimelb.edu.au).

This work was supported by the Australian Research Council under grant DP250101763 and the United States Air Force Office of Scientific Research under Grant No. FA2386-24-1-4014.

[1]https://github.com/mingu6/FilterDDP.jl

II. Discrete-time Optimal Control

In this section, we provide some background on equality constrained discrete-time optimal control problems (OCPs).

A. Problem Formulation

We consider OCPs with a finite-horizon length N with equality constraints, which can be expressed in the form

$$\begin{aligned} \underset{\mathbf{x},\mathbf{u}}{\text{minimise}} \quad & J(x_{1:N}, u_{1:N}) \coloneqq \textstyle\sum_{t=1}^{N} \ell(x_t, u_t) \\ \text{subject to} \quad & x_1 = \hat{x}_1, \\ & x_{t+1} = f(x_t, u_t) \quad \text{for } t \in [N-1], \\ & c(x_t, u_t) = 0 \quad \text{for } t \in [N], \end{aligned} \tag{1}$$

where $x_{1:N}$ and $u_{1:N}$ are a trajectory of states and control inputs, respectively, and we assume $x_t \in \mathbb{R}^{n_x}$ and $u_t \in \mathbb{R}^{n_u}$ for all t. The costs are denoted by $\ell : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}$ and the constraints are denoted by $c : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_c}$ where $n_c \leq n_u$. The mapping $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_x}$ captures the discrete time system dynamics and $\hat{x}_1$ is the initial state. We require that ℓ, f, c are twice continuously differentiable.

B. Optimality Conditions

The Lagrangian of the OCP in (1) is given by

$$\begin{aligned} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) \coloneqq \lambda_1^\top (\hat{x}_1 - x_1) + \textstyle\sum_{t=1}^{N} L(x_t, u_t, \phi_t) \\ + \textstyle\sum_{t=1}^{N-1} \lambda_{t+1}^\top (f(x_t, u_t) - x_{t+1}), \end{aligned} \tag{2}$$

where $\phi_{1:N}$ and $\boldsymbol{\lambda} \coloneqq \lambda_{1:N}$ are the dual variables for the equality and dynamics constraints in (1), respectively. Furthermore, $w_t = (x_t, u_t, \phi_t)$, $\mathbf{w} \coloneqq w_{1:N}$ and finally, $L(x_t, u_t, \phi_t) \coloneqq \ell(x_t, u_t) + \phi_t^\top c(x_t, u_t)$.

The first-order optimality conditions for (1) (also known as Karush-Kuhn-Tucker (KKT) conditions), necessitate that trajectories $x_{1:N}, u_{1:N}$ can only be a local solution of (1) if there exist dual variables $\lambda_{1:N}$ and $\phi_{1:N}$ such that

$$\nabla_{x_t} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = L_x^t - \lambda_t^\top + \lambda_{t+1}^\top f_x^t = 0, \tag{3a}$$

$$\nabla_{u_t} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = L_u^t + \lambda_{t+1}^\top f_u^t = 0, \tag{3b}$$

$$\nabla_{\lambda_t} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = \begin{cases} (\hat{x}_1 - x_t)^\top = 0 & t = 1 \\ (f(x_{t-1}, u_{t-1}) - x_t)^\top = 0 & t > 1 \end{cases}, \tag{3c}$$

$$\nabla_{\phi_t} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = c(x_t, u_t)^\top = 0, \tag{3d}$$

where $L_x^t, L_u^t, f_x^t, f_u^t$ are partial derivatives of L and f at w_t and noting that for $t = N$, we implicitly set $\lambda_{t+1} = 0$ since there are no dynamics constraints for step $t = N$.

III. Background and Derivation of DDP

We now provide some background on the derivation of our constrained differential dynamic programming algorithm.

A. Dynamic Programming for Optimal Control

Optimal control problems are amenable to being solved using *dynamic programming* (DP), which relies on Bellman's principle of optimality [30]. Concretely, for the OCP in (1), the optimality principle at time step $t \in [N]$ yields

$$\begin{aligned} V_t^\star(x_t) \coloneqq \min_{u_t} \quad & \ell(x_t, u_t) + V_{t+1}^\star(f(x_t, u_t)) \\ \text{s.t.} \quad & c(x_t, u_t) = 0, \end{aligned} \tag{4}$$

where $V_t^\star$ is the *optimal value function* with boundary condition $V_{N+1}^\star(x) \coloneqq 0$. By duality, (4) is equivalent to

$$V_t^\star(x_t) \coloneqq \min_{u_t} \max_{\phi_t} L(x_t, u_t, \phi_t) + V_{t+1}^\star(f(x_t, u_t)), \tag{5}$$

implying that, 1) we can decompose (1) into a sequence of sub-problems, which are solved recursively backwards in time from $t = N$ and, 2) $V_t^\star(x_t)$ is the optimal value of the Lagrangian of the subsequence starting at step t and state x_t.

B. Differential Dynamic Programming

A core idea used to derive the FilterDDP algorithm, analogous to [31], is to replace the intractable *optimal* value function $V_t^\star$ with a tractable *sub-optimal* V^t. As a result, the intractable problems defined in (5) are replaced with

$$\min_{u_t} \max_{\phi_t} Q^t(x_t, u_t, \phi_t), \tag{6}$$

where $Q^t(x_t, u_t, \phi_t) \coloneqq L(x_t, u_t, \phi_t) + V^{t+1}(f(x_t, u_t))$.

The FilterDDP algorithm alternates between a backward pass and forward pass phase as in unconstrained DDP [13]. The backward pass applies a *perturbed* Newton step to (6) to determine the nonlinear update rule applied in the forward pass. We defer formally defining V^t to Sec. IV-E.

IV. Filter Differential Dynamic Programming

We now describe the full FilterDDP algorithm. FilterDDP generates a sequence of iterates $\{(\mathbf{w}_k, \boldsymbol{\lambda}_k)\}$ by alternating between a *backward pass* and *forward pass*.

A. Termination Criterion

Let $w_t \coloneqq (x_t, u_t, \phi_t)$ and λ_t together, represent the current iterate at time step t. The optimality error used to determine convergence, letting $c^t \coloneqq c(x_t, u_t)$, is defined to be

$$E(\mathbf{w}, \boldsymbol{\lambda}) \coloneqq \max_t \max \left\{ \|\nabla_{u_t} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda})\|_\infty, \|c^t\|_\infty \right\}. \tag{7}$$

The algorithm terminates successfully if $E(\mathbf{w}, \boldsymbol{\lambda}) < \epsilon_{\text{tol}}$ for a user-defined tolerance $\epsilon_{\text{tol}} > 0$. We omit $\nabla_{x_t} \mathcal{L}$ in (7) since for the iterates under the FilterDDP algorithm, it will be 0 under the definition of λ_t to be presented in (14a).

B. Backward Pass

The backward pass computes the first and perturbed second derivatives of V^t around the current iterate $\bar{\mathbf{w}}$, denoted by $\bar{V}_x^t \in \mathbb{R}^{1 \times n_x}$ and $P_t \in \mathbb{R}^{n_x \times n_x}$, backwards in time. Next, an update rule is derived using a *perturbed* Newton step applied to the stationarity condition of (6). The perturbation allows us to establish local convergence of FilterDDP in Sec. V.

Notation: For a function $h \in \{\ell, f, c, L, Q^t\}$ and $y \in \{x, u\}$, let $\bar{h}_y^t \coloneqq \nabla_y h(\bar{x}_t, \bar{u}_t, \bar{\phi}_t)$. We use similar notation for second derivatives of h. Furthermore, let $\bar{h}^t \coloneqq h(\bar{x}_t, \bar{u}_t, \bar{\phi}_t)$.

The backward pass sets the boundary conditions

$$\bar{\lambda}_{N+1} = 0_{1 \times n_x}, \;\; \bar{V}_x^{N+1} = 0_{1 \times n_x}, \;\; P_{N+1} = 0_{n_x \times n_x}, \tag{8}$$

and proceeds backwards in time, alternating between the perturbed Newton step and updating $\bar{V}_x^t$, P_t and $\bar{\lambda}_t$.

a) Update Rule: We first present the unperturbed Newton step to the stationarity condition of (6), given by

$$\nabla_{u_t} Q^t(x_t, u_t, \phi_t) = 0, \quad c(x_t, u_t) = 0. \tag{9}$$

Let $\delta w_t \coloneqq (\delta x_t, \delta u_t, \delta \phi_t)$ represent an update to $\bar{w}_t$.

Newton's method applied to (9) yields

$$\begin{bmatrix} \bar{Q}^t_{uu} & A_t \\ A_t^\top & 0 \end{bmatrix} \begin{bmatrix} \delta u_t \\ \delta \phi_t \end{bmatrix} = - \begin{bmatrix} (\bar{Q}^t_u)^\top \\ \bar{c}^t \end{bmatrix} - \begin{bmatrix} \bar{Q}^t_{ux} \\ \bar{c}^t_x \end{bmatrix} \delta x_t, \quad (10)$$

where $A_t \coloneqq (\bar{c}^t_u)^\top$, $\bar{Q}^t_u = \bar{L}^t_u + \bar{V}^{t+1}_x \bar{f}^t_u$,

$$\bar{Q}^t_{uu} = \bar{L}^t_{uu} + (\bar{f}^t_u)^\top \bar{V}^{t+1}_{xx} \bar{f}^t_u + \bar{V}^{t+1}_x \cdot \bar{f}^t_{uu}, \quad (11a)$$
$$\bar{Q}^t_{ux} = \bar{L}^t_{ux} + (\bar{f}^t_u)^\top \bar{V}^{t+1}_{xx} \bar{f}^t_x + \bar{V}^{t+1}_x \cdot \bar{f}^t_{ux}, \quad (11b)$$
$$\bar{Q}^t_{xx} = \bar{L}^t_{xx} + (\bar{f}^t_x)^\top \bar{V}^{t+1}_{xx} \bar{f}^t_x + \bar{V}^{t+1}_x \cdot \bar{f}^t_{xx}, \quad (11c)$$

and $\cdot$ denotes a tensor contraction along the first dimension. Equation (10) implies that $\delta u_t = \alpha_t + \beta_t \delta x_t$ and $\delta \phi_t = \psi_t + \omega_t \delta x_t$ for some parameters $\alpha_t, \beta_t, \psi_t, \omega_t$. To determine the update rule, FilterDDP applies the perturbed Newton step given by

$$\underbrace{\begin{bmatrix} H_t + \delta_w I & A_t \\ A_t^\top & -\delta_c I \end{bmatrix}}_{K_t} \begin{bmatrix} \alpha_t & \beta_t \\ \psi_t & \omega_t \end{bmatrix} = - \begin{bmatrix} (\bar{Q}^t_u)^\top & B_t \\ \bar{c}^t & \bar{c}^t_x \end{bmatrix}, \quad (12)$$

where H_t, B_t, C_t are given by

$$H_t \coloneqq \bar{L}^t_{uu} + (\bar{f}^t_u)^\top P_{t+1} \bar{f}^t_u + \bar{\lambda}_{t+1} \cdot \bar{f}^t_{uu}, \quad (13a)$$
$$B_t \coloneqq \bar{L}^t_{ux} + (\bar{f}^t_u)^\top P_{t+1} \bar{f}^t_x + \bar{\lambda}_{t+1} \cdot \bar{f}^t_{ux}, \quad (13b)$$
$$C_t \coloneqq \bar{L}^t_{xx} + (\bar{f}^t_x)^\top P_{t+1} \bar{f}^t_x + \bar{\lambda}_{t+1} \cdot \bar{f}^t_{xx} \quad (13c)$$

and the recursive update of P_t is described in (14b).

Motivated by global convergence [25], we set $\delta_w, \delta_c \geq 0$ following Alg. IC in [27] so that K_t has an inertia[2] of $(n_u, n_c, 0)$. We factorize K_t and recover its inertia using an LDLT factorization [32], [33].

b) Updating the Value Function Approximation: After solving (12), $\bar{V}^t_x, P_t$ and $\bar{\lambda}_t$ are updated according to

$$\bar{V}^t_x = \bar{Q}^t_x + \bar{Q}^t_u \beta_t + (\bar{c}^t)^\top \omega_t, \quad \bar{\lambda}_t^\top = \bar{L}^t_x + \bar{\lambda}^\top_{t+1} \bar{f}^t_x \quad (14a)$$
$$P_t = C_t + \beta_t^\top H_t \beta_t + B_t^\top \beta_t + \beta_t^\top B_t. \quad (14b)$$

The time step is then decremented, i.e., $t \leftarrow t-1$ and *a)* is repeated. We will defer defining the value function V^t from which $\bar{V}^t_x$ and $\bar{V}^t_{xx}$ are derived until Sec. IV-E, since the forward pass in Sec. IV-C must be described first.

c) Differences to prior works e.g., [22], [23], [31]: Both the perturbed Newton step (12) and replacing $\|\bar{Q}^t_u\|_\infty$ with (7) for the termination criteria are novel inclusions of FilterDDP, motivated by the convergence result in Sec. V.

C. Forward Pass

The forward pass generates trial points for the next iterate. For a step size $\gamma \in (0, 1]$, the trial point is given by applying

$$\begin{aligned} x^+_{t+1} &\coloneqq f(x^+_t, u^+_t(x^+_t, \gamma)), \quad x^+_1 = \hat{x}_1, \\ u^+_t(x^+_t, \gamma) &\coloneqq \bar{u}_t + \gamma \alpha_t + \beta_t (x^+_t - \bar{x}_t), \\ \phi^+_t(x^+_t, \gamma) &\coloneqq \bar{\phi}_t + \gamma \psi_t + \omega_t (x^+_t - \bar{x}_t), \end{aligned} \quad (15)$$

forward in time starting from $t = 1$. A trial point $\mathbf{w}^+(\gamma)$, where $w^+_t(\gamma) \coloneqq (x^+_t, u^+_t(x^+_t, \gamma), \phi^+_t(x^+_t, \gamma))$ is accepted as the next iterate if step acceptance criteria described below are satisfied. The backtracking line search procedure sets $\gamma = 1$, setting $\gamma \leftarrow \frac{1}{2}\gamma$ if the trial point $\mathbf{w}^+(\gamma)$ is not accepted.

b) Step acceptance: A trial point must yield a sufficient decrease in *either* the Lagrangian of (1) or the constraint violation compared to the current iterate, concretely,

$$\mathcal{L}(\mathbf{w}) \coloneqq \sum_{t=1}^N L(x_t, u_t, \phi_t), \quad \theta(\mathbf{w}) \coloneqq \sum_{t=1}^N \|c^t\|_1. \quad (16)$$

We adopt the sufficient decrease condition given by

$$\theta(\mathbf{w}^+(\gamma)) \leq (1 - \gamma_\theta)\theta(\bar{\mathbf{w}}) \quad \text{or} \quad (17a)$$
$$\mathcal{L}(\mathbf{w}^+(\gamma)) \leq \mathcal{L}(\bar{\mathbf{w}}) - \gamma_{\mathcal{L}} \theta(\bar{\mathbf{w}}), \quad (17b)$$

for constants $\gamma_\theta, \gamma_{\mathcal{L}} \in (0, 1)$. Furthermore, following [25], [27], (17) is replaced by enforcing a sufficient decrease condition on $\mathcal{L}$ if both $\theta(\bar{\mathbf{w}}) < \theta_{\min}$ for some small $\theta_{\min}$ and the *switching condition*

$$\gamma m < 0 \quad \text{and} \quad (-\gamma m)^{s_{\mathcal{L}}} \gamma^{1 - s_{\mathcal{L}}} > \delta (\theta(\bar{\mathbf{w}}))^{s_\theta} \quad (18)$$

holds, where $\delta > 0$, $s_\theta > 1$, $s_{\mathcal{L}} \geq 1$ are constants and

$$m \coloneqq \sum_{t=1}^N \left(\bar{Q}^t_u \alpha_t + \psi_t^\top \bar{c}^t \right). \quad (19)$$

The switching condition (18) is motivated by the global convergence analysis of [25]. The aforementioned decrease condition on $\mathcal{L}$ is given by

$$\mathcal{L}(\mathbf{w}^+(\gamma)) \leq \mathcal{L}(\bar{\mathbf{w}}) + \eta_{\mathcal{L}} \gamma m \quad (20)$$

for some small $\eta_{\mathcal{L}} > 0$. Accepted iterates which satisfy (18) and (20) are called $\mathcal{L}$-type iterations [27].

FilterDDP maintains a *filter*, denoted by $\mathcal{F} \subseteq \{(\theta, \mathcal{L}) \in \mathbb{R}^2 : \theta \geq 0\}$. The set $\mathcal{F}$ defines a "taboo" region for iterates, and a trial point is rejected if $(\theta^+, \mathcal{L}^+) \in \mathcal{F}$. We initialise the filter with $\mathcal{F} = \{(\theta, \mathcal{L}) \in \mathbb{R}^2 : \theta \geq \theta_{\max}\}$ for some $\theta_{\max}$. After a trial point $\mathbf{w}^+(\gamma)$ is accepted, the filter is augmented if either (18) or (20) do not hold, using

$$\begin{aligned} \mathcal{F}^+ \coloneqq \mathcal{F} \cup \big\{ (\theta, \mathcal{L}) \in \mathbb{R}^2 : \theta \geq (1 - \gamma_\theta)\theta(\bar{\mathbf{w}}), \\ \mathcal{L} \geq \mathcal{L}(\bar{\mathbf{w}}) - \gamma_{\mathcal{L}} \theta(\bar{\mathbf{w}}) \big\}. \end{aligned} \quad (21)$$

a) Differences to [31], [15]: In contrast to existing non-AL DDP methods, FilterDDP uses the *Lagrangian* within the step acceptance criteria instead of the cost. We motivate this by formally showing in Sec. IV-E that (20) is analogous to the well-known Armijo condition [24, pg. 33] . We note that global convergence of line-search filter methods for general nonlinear programs is preserved under this change [25, Sec. 4.1].

D. Complete FilterDDP Algorithm

Below, we present the full FilterDDP algorithm.

Algorithm 1:

Given: Starting point $\mathbf{w}_0$; $\theta_{\max} \in (0, \infty]$; $\gamma_\theta, \gamma_{\mathcal{L}} \in (0, 1)$; $\epsilon_{\text{tol}}, \delta, \gamma^{\min}, \text{max\_iters} > 0$; $s_\theta > 1$; $s_{\mathcal{L}} \geq 1$; $\eta_{\mathcal{L}} \in (0, \frac{1}{2})$.

1. *Initialise.* Initialise the iteration counter $k \leftarrow 0$ and filter $\mathcal{F}_k \coloneqq \{(\theta, \mathcal{L}) \in \mathbb{R}^2 : \theta \geq \theta_{\max}\}$.
2. *Backward pass.* Set value function boundary conditions, i.e., (8) and set time counter $t \leftarrow N$.

[2]The inertia is the number of positive, negative and zero eigenvalues.

2.1. If $t = 0$, go to step 3.
2.2. Compute blocks in (12), i.e., H_t, B_t, etc. using $\mathbf{w}_k$, $\boldsymbol{\lambda}_k$ and solve for $\alpha_t, \beta_t, \psi_t, \omega_t$. If the matrix in (12) is too ill conditioned, terminate the algorithm.
2.3. Update $\bar{V}_x^t$, P_t, $\bar{\lambda}_t$ using (14).
2.4. Set $t \leftarrow t - 1$ and go to 2.1.
3. *Check convergence.* If $E(\mathbf{w}_k, \boldsymbol{\lambda}_k) < \epsilon_{\text{tol}}$ then STOP (success). If $k = \text{max\_iters}$ then STOP (failure).
4. *Backtracking line search.*
4.1. *Initialise line search.* Set $l \leftarrow 0$ and $\gamma_l = 1$.
4.2. *Compute new trial point.* If the trial step size γ_l becomes too small, i.e., $\gamma_l < \gamma^{\min}$, terminate the algorithm. Otherwise, compute the new trial point $\mathbf{w}^+(\gamma_l)$ using the forward pass, i.e., (15).
4.3. *Check acceptability to the filter.* If (17) does not hold, reject the trial step size and go to step 5.
4.4. *Check sufficient decrease against the current iterate.*
4.4.1. *Case* I: γ_l is a $\mathcal{L}$-step size (i.e., (18) holds). If the Armijo condition (20) holds, accept the trial step and go to step 6. Otherwise, go to step 5.
4.4.2. *Case* II: γ_l is not a $\mathcal{L}$-step size, (i.e., (18) is not satisfied): If (17) holds, accept the trial step and go to step 6. Otherwise, go to step 5.
5. *Choose new trial step size.* Choose $\gamma_{l+1} = \frac{1}{2}\gamma_l$, set $l \leftarrow l + 1$ and go back to step 4.2.
6. *Accept trial point.* Set $\gamma := \gamma_l$ and $\mathbf{w}_{k+1} := \mathbf{w}_k^+(\gamma)$.
7. *Augment filter if necessary.* If k is not a $\mathcal{L}$-type iteration, augment the filter using (21); otherwise leave the filter unchanged, i.e., set $\mathcal{F}_{k+1} := \mathcal{F}_k$.
8. *Continue with next iteration.* Increase the iteration counter $k \leftarrow k + 1$ and go back to step 2.

E. Definition of the Sub-Optimal Value Function V^t

We conclude the section by presenting a convenient candidate for the value function V^t whose derivatives satisfies (14). In doing so, we also provide intuition on (14) and (20).

Theorem 1: A value function V^t which has associated first and second-order derivatives given by (14a) and $\bar{V}_{xx}^t = \bar{Q}_{xx}^t + \beta_t^\top \bar{Q}_{uu}^t \beta_t + \bar{Q}_{xu}^t \beta_t + \beta_t^\top \bar{Q}_{ux}^t$ (i.e., unperturbed (14b)), respectively, is given by $V^{N+1}(x, \gamma) := 0$, and

$$V^t(x, \gamma) := Q^t(x, u_t^+(x, \gamma), \phi_t^+(x, \gamma)). \tag{22}$$

Note that $V^1(\hat{x}_1, \gamma) = \mathcal{L}(\mathbf{w}^+(\gamma))$, i.e., the updated $\mathcal{L}$, and that $\bar{V}_x^t = \nabla_x V^t(\bar{x}_t, 0)$ and $\bar{V}_{xx}^t = \nabla_{xx}^2 V^t(\bar{x}_t, 0)$.

Proof: For the first derivative,

$$\begin{aligned}\nabla_x V^t(\bar{x}_t, 0) &\overset{(22)}{=} \bar{Q}_x^t + \bar{Q}_u^t \nabla_x u_t^+(\bar{x}_t, 0) + \bar{Q}_\phi^t \phi_t^+(\bar{x}_t, 0) \\ &\overset{(15)}{=} \bar{Q}_x^t + \bar{Q}_u^t \beta_t + (\bar{c}^t)^\top \omega_t = \bar{V}_x^t.\end{aligned}$$

For the second deriative, let $y(x, \gamma) := (x, u_t^+(x, \gamma), \phi_t^+(x, \gamma))$. It follows that

$$\begin{aligned}\nabla_{xx}^2 V^t(\bar{x}_t, 0) &= \nabla_{xx}^2 Q^t(y(\bar{x}_t, 0)) \\ &= (\nabla_x y(\bar{x}_t, 0))^\top \nabla_{yy}^2 Q^t(y(\bar{x}_t, 0)) \nabla_x y(\bar{x}_t, 0) \\ &\quad + \nabla_y Q^t(y(\bar{x}_t, 0)) \cdot \underbrace{\nabla_{xx}^2 y(\bar{x}_t, 0)}_{=0} \\ &= \begin{bmatrix} I & \beta_t^\top & \omega_t^\top \end{bmatrix} \begin{bmatrix} \bar{Q}_{xx}^t & \bar{Q}_{xu}^t & (\bar{c}_x^t)^\top \\ \bar{Q}_{ux}^t & \bar{Q}_{uu}^t & A_t \\ \bar{c}_x^t & A_t^\top & 0 \end{bmatrix} \begin{bmatrix} I \\ \beta_t \\ \omega_t \end{bmatrix} \\ &= \bar{Q}_{xx}^t + \bar{Q}_{xu}^t \beta_t + (\bar{c}_x^t)^\top \omega_t + \beta_t^\top \bar{Q}_{ux}^t \\ &\quad + \beta_t^\top \bar{Q}_{uu}^t \beta_t + \beta_t^\top A_t \omega_t + \omega_t^\top \bar{c}_x^t + \omega_t^\top A_t^\top \beta_t \\ &\overset{(12)}{=} \bar{Q}_{xx}^t + \beta_t^\top \bar{Q}_{uu}^t \beta_t + \bar{Q}_{ux}^t \beta_t + \beta_t^\top \bar{Q}_{ux}^t \\ &= \bar{V}_{xx}^t,\end{aligned}$$

where (12) is used through the relation $A_t^\top \beta_t = -\bar{c}_x^t$. ∎

Corollary 1: $m = \frac{d}{d\gamma}\mathcal{L}(\mathbf{w}^+(0))$, i.e., m is similar to the directional derivative in the Armijo condition [24, pg. 33].

Proof: Differentiating (22) w.r.t. γ and evaluating at $\gamma = 0$ and $x = \bar{x}$ yields $\bar{V}_\gamma^t = \bar{Q}_u^t \alpha_t + \psi_t^\top \bar{c}^t + \bar{V}_\gamma^{t+1}$. Applying an inductive argument yields $\bar{V}_\gamma^1 = m$. Since $V^1(\hat{x}_1, \gamma) = \mathcal{L}(\mathbf{w}^+(\gamma))$, it follows that $\bar{V}_\gamma^1 = \frac{d}{d\gamma}\mathcal{L}(\mathbf{w}^+(0))$. ∎

Corollary 1 motivates using $\mathcal{L}$ for step acceptance in (20).

V. Convergence Analysis of FilterDDP

In this section, we establish the local quadratic convergence of FilterDDP within a sufficiently small neighbourhood of a solution. We simplify our analysis by assuming all iterates with full step size $\gamma = 1$ are accepted by the filter criteria in Sec. IV-C, leaving the more general analysis (with a second-order correction step [26]) for future work. We first establish that an optimal point $\mathbf{w}^\star$ satisfies KKT conditions (3) if and only if it is a fixed point of FilterDDP. We then prove local quadratic convergence to $\mathbf{w}^\star$.

Assumptions 1. For all $t \in [N]$ there exists an open convex neighbourhood $\mathcal{N}_t$ containing $w_t^\star$ and the current iterate $\bar{w}_t$, such that for all $w_t \in \mathcal{N}_t$ and all $t \in [N]$,

(1A) functions ℓ, f, c and their first derivatives are Lipschitz continuously differentiable;
(1B) the minimum singular value of A_t is uniformly bounded away from zero; and
(1C) $\xi^\top H_t \xi > 0$ for all ξ in the null space of A_t.

Assumption (1C) ensures that no inertia correction is applied and that the iteration matrix K_t is non-singular. Furthermore, Assumption (1B) holds under suitable constraint qualifications on (1) (e.g., LICQ [24, Definition 12.4]).

Theorem 2: Let $\bar{\mathbf{w}}$ be a feasible point which satisfies $\bar{x}_{t+1} = f(\bar{x}_t, \bar{u}_t)$, $\bar{x}_1 = \hat{x}_1$ and $c(\bar{x}_t, \bar{u}_t) = 0$. Then $\bar{\mathbf{w}}$ is a fixed point of FilterDDP (i.e., $\alpha_t = 0$, $\psi_t = 0$ for all $t \in [N]$) if and only if it also satisfies KKT conditions (3).

Proof: Suppose $\bar{\mathbf{w}}$ is a fixed point of FilterDDP. Then by non-singularity of K_t, $\bar{Q}_u^t = 0$, $\bar{c}^t = 0$. By (14a) and the definition of $\bar{Q}_u^t$, $\bar{V}_x^t = \bar{\lambda}_t$ for all $t \in [N]$. It follows that $\nabla_{u_t}\mathcal{L}(\bar{\mathbf{w}}, \bar{\boldsymbol{\lambda}}) = \bar{L}_u^t + \bar{\lambda}_{t+1}\bar{f}_u^t = \bar{Q}_u^t = 0$. Substituting the update rule for $\bar{\lambda}_t$ into (3a) yields $\nabla_{x_t}\mathcal{L}(\bar{\mathbf{w}}, \bar{\boldsymbol{\lambda}}) = 0$. The remaining conditions in (3) are satisfied by assumption.

Now suppose that $\bar{\mathbf{w}}$ satisfies (3). Then $\nabla_{u_t}\mathcal{L}(\bar{\mathbf{w}},\bar{\boldsymbol{\lambda}}) = \bar{L}_u^t + \bar{\lambda}_{t+1}^\top \bar{f}_u^t = 0$ and $\bar{c}^t = 0$. Since $\bar{c}^N = 0$ and $\bar{L}_u^N = 0$, by (14a) it follows that $\bar{V}_x^N = \bar{\lambda}_N$. Since $\bar{c}^{N-1} = 0$ and $\bar{L}_u^{N-1} + \bar{\lambda}_N^\top \bar{f}_u^{N-1} = 0$, by (14a) it follows that $\bar{V}_x^{N-1} = \bar{\lambda}_{N-1}$ and consequently $\bar{Q}_u^{N-1} = 0$. An inductive argument yields $\bar{Q}_u^t = 0$ for all $t \in [N]$, which with $\bar{c}^t = 0$ and K_t non-singular implies that $\bar{\mathbf{w}}$ is a fixed point of FilterDDP. ■

We now establish local quadratic convergence. Denote an optimal point by $\mathbf{w}^\star$ and let R abbreviate $O(\|\mathbf{w}^\star - \bar{\mathbf{w}}\|^2)$.

Proposition 1: Let $\lambda^{N+1}(\mathbf{w}) = 0_{n_x}$ and for $t \in [N]$,

$$\lambda^t(\mathbf{w})^\top \coloneqq \nabla_x L(x_t,u_t,\phi_t) + \lambda^{t+1}(\mathbf{w})^\top \nabla_x f(x_t,u_t). \quad (23)$$

Note that $\lambda^t(\bar{\mathbf{w}}) = (\bar{\lambda}_t)^\top$. Under Assumptions 1,

$$\lambda^t(\mathbf{w}^\star) = (\bar{V}_x^t)^\top + P_t(x_t^\star - \bar{x}_t) + R. \quad (24)$$

Proof: We proceed with an inductive argument. The base case $t = N+1$ holds since $\lambda^{N+1} = 0$, $\bar{V}_x^{N+1} = 0$ and $P_{N+1} = 0$. Suppose (24) holds for some step $t+1$. By Ass. (1A), we can apply Taylor's theorem to f and c, yielding

$$x_{t+1}^\star = \bar{x}_{t+1} + \bar{f}_x^t(x_t^\star - \bar{x}_t) + \bar{f}_u^t(u_t^\star - \bar{u}_t) + R, \quad (25)$$

$$0 = c^t(x_t^\star,u_t^\star) = \bar{c}^t + A_t^\top(u_t^\star - \bar{u}_t) + \bar{c}_x^t(x_t^\star - \bar{x}_t) + R. \quad (26)$$

In addition, by Ass. (1A), the derivatives of λ^t exist (locally) and are Lipschitz, given by $\bar{\lambda}^t_{w_{t+1:N}} = (\bar{f}_x^t)^\top \bar{\lambda}^{t+1}_{w_{t+1:N}}$ and

$$\bar{\lambda}_{y_t}^t = \bar{L}_{xy}^t + \bar{\lambda}^{t+1}\cdot \bar{f}_{xy}^t, \quad y \in \{x,u\}, \quad \bar{\lambda}_\phi^t = (\bar{c}_x^t)^\top, \quad (27)$$

also noting that derivatives of λ^t w.r.t w_s for $s < t$ are zero.

Applying Taylor's theorem to (23) for step t yields

$$\begin{aligned}
\lambda^t(\mathbf{w}^\star) &= \bar{\lambda}^t + \bar{\lambda}^t_{\mathbf{w}}(\mathbf{w}^\star - \bar{\mathbf{w}}) + R \\
&\overset{(23)}{=} \bar{\lambda}^t + \bar{\lambda}^t_{w_t}(w_t^\star - \bar{w}_t) \\
&\quad + (\bar{f}_x^t)^\top \bar{\lambda}^{t+1}_{\mathbf{w}}(\mathbf{w}^\star - \bar{\mathbf{w}}) + R \\
&\overset{(23)}{=} (\bar{L}_x^t)^\top + \bar{\lambda}^t_{w_t}(w_t^\star - \bar{w}_t) \\
&\quad + (\bar{f}_x^t)^\top \lambda^{t+1}(\mathbf{w}^\star) + R \\
&\overset{(24),(25)}{=} (\bar{Q}_x^t)^\top + \bar{\lambda}^t_{w_t}(w_t^\star - \bar{w}_t) + (\bar{f}_x^t)^\top P_{t+1}(\\
&\quad \bar{f}_x^t(x_t^\star - \bar{x}_t) + \bar{f}_u^t(u_t^\star - \bar{u}_t)) + R \\
&\overset{(27),(13)}{=} (\bar{Q}_x^t)^\top + C_t(x_t^\star - \bar{x}_t) + B_t^\top(u_t^\star - \bar{u}_t) \\
&\quad + (\bar{c}_x^t)^\top(\phi_t^\star - \bar{\phi}_t) + R.
\end{aligned} \quad (28)$$

By Theorem 2, $\nabla_{u_t}\mathcal{L}(\mathbf{w}^\star, \boldsymbol{\lambda}(\mathbf{w}^\star)) = 0$. Applying Taylor's theorem to this expression around $\bar{\mathbf{w}}$ yields,

$$\begin{aligned}
0 &= \bar{\mathcal{L}}^t_{u_t} + \bar{\mathcal{L}}^t_{u_t w_t}(w_t^\star - \bar{w}_t) \\
&\quad + \bar{\mathcal{L}}_{u_t w_{t+1:N}}(w^\star_{t+1:N} - \bar{w}_{t+1:N}) + R \\
&\overset{(3b)}{=} (\bar{L}_u^t)^\top + (\bar{f}_u^t)^\top \bar{\lambda}_{t+1} + \bar{\mathcal{L}}^t_{u_t w_t}(w_t^\star - \bar{w}_t) \\
&\quad + (\bar{f}_u^t)^\top\big(\bar{\lambda}^{t+1}_{w_{t+1:N}}(w^\star_{t+1:N} - \bar{w}_{t+1:N})\big) + R \\
&\overset{(24),(25)}{=} (\bar{Q}_u^t)^\top + \bar{\mathcal{L}}^t_{u_t w_t}(w_t^\star - \bar{w}_t) + (\bar{f}_u^t)^\top P_{t+1}(\\
&\quad \bar{f}_x^t(x_t^\star - \bar{x}_t) + \bar{f}_u^t(u_t^\star - \bar{u}_t)) + R \\
&\overset{(3b)}{=} R + (\bar{Q}_u^t)^\top + A_t(\phi_t^\star - \bar{\phi}_t) \\
&\quad + (\bar{L}^t_{uu} + (\bar{f}_u^t)^\top P_{t+1}\bar{f}_u^t + \bar{\lambda}_{t+1}\cdot \bar{f}^t_{uu})(u_t^\star - \bar{u}_t) \\
&\quad + (\bar{L}^t_{ux} + (\bar{f}_u^t)^\top P_{t+1}\bar{f}_x^t + \bar{\lambda}_{t+1}\cdot \bar{f}^t_{ux})(x_t^\star - \bar{x}_t) \\
&\overset{(13)}{=} (\bar{Q}_u^t)^\top + H_t(u_t^\star - \bar{u}_t) + B_t(x_t^\star - \bar{x}_t) \\
&\quad + A_t(\phi_t^\star - \bar{\phi}_t) + R.
\end{aligned} \quad (29)$$

Rearranging (29) for $(\bar{Q}_u^t)^\top$, substituting the resulting expression into (14a) and substituting the resulting expression for $(\bar{Q}_x^t)^\top$ into (28) yields

$$\begin{aligned}
\lambda^t(\mathbf{w}^\star) &= (\bar{V}_x^t)^\top + (C_t + \beta_t^\top B_t)(x_t^\star - \bar{x}_t) - \omega_t^\top \bar{c}^t \\
&\quad + (\beta^\top H_t + B_t^\top)(u_t^\star - \bar{u}_t) \\
&\quad + (A_t^\top \beta_t + \bar{c}_x^t)^\top(\phi_t^\star - \bar{\phi}) + R \\
&\overset{(12)}{=} (\bar{V}_x^t)^\top - \omega_t^\top A_t^\top(u_t^\star - \bar{u}_t) \\
&\quad + (C_t + \beta_t^\top B_t)(x_t^\star - \bar{x}_t) - \omega_t^\top \bar{c}^t + R \\
&\overset{(26)}{=} (\bar{V}_x^t)^\top + (C_t + \beta_t^\top B_t + \omega_t^\top \bar{c}_x^t)(x_t^\star - \bar{x}_t) + R \\
&\overset{(12)}{=} (\bar{V}_x^t)^\top + (C_t + \beta_t^\top B_t \\
&\quad - \omega_t^\top A_t^\top \beta_t)(x_t^\star - \bar{x}_t) + R \\
&\overset{(12),(14b)}{=} (\bar{V}_x^t)^\top + P_t(x_t^\star - \bar{x}_t) + R,
\end{aligned} \quad (30)$$

as required. ■

Remark 1: Stacking (29) and (26) yields the identity

$$\begin{bmatrix}(\bar{Q}_u^t)^\top \\ \bar{c}^t\end{bmatrix} = -K_t\begin{bmatrix}u_t^\star - \bar{u}_t \\ \phi_t^\star - \bar{\phi}_t\end{bmatrix} - \begin{bmatrix}B_t \\ \bar{c}_x^t\end{bmatrix}(x_t^\star - \bar{x}_t) + R. \quad (31)$$

Finally, we present our main result, which shows that when the current iterate is sufficiently close to an optimal point $\mathbf{w}^\star$, FilterDDP converges at a quadratic rate. Let $\mathbf{w}^+$ be shorthand for $\mathbf{w}^+(1)$, i.e., a full forward pass step where $\gamma = 1$.

Theorem 3: Under Assumptions 1, there exists positive scalars ϵ and M such that if $\|\mathbf{w}^\star - \bar{\mathbf{w}}\| < \epsilon$,

$$\|\mathbf{w}^\star - \mathbf{w}^+\| \le M\|\mathbf{w}^\star - \bar{\mathbf{w}}\|^2, \quad (32)$$

and furthermore,

$$\|\mathbf{w}^\star - \mathbf{w}^+\| < \|\mathbf{w}^\star - \bar{\mathbf{w}}\|. \quad (33)$$

Proof: Assumptions (1C) and (1B) ensures K_t in (12) is non-singular for all $t \in [N]$. The forward pass yields

$$\begin{aligned}
\begin{bmatrix}u_t^+ - \bar{u}_t \\ \phi_t^+ - \bar{\phi}_t\end{bmatrix} &\overset{(15)}{=} \begin{bmatrix}\alpha_t \\ \psi_t\end{bmatrix} + \begin{bmatrix}\beta_t \\ \omega_t\end{bmatrix}(x_t^+ - \bar{x}_t) \\
&\overset{(12)}{=} -(K_t)^{-1}\begin{bmatrix}(\bar{Q}_u^t)^\top \\ \bar{c}^t\end{bmatrix} - (K_t)^{-1}\begin{bmatrix}B_t \\ \bar{c}_x^t\end{bmatrix}(x_t^+ - \bar{x}_t) \\
&\overset{(31)}{=} \begin{bmatrix}u_t^\star - \bar{u}_t \\ \phi_t^\star - \bar{\phi}_t\end{bmatrix} - (K_t)^{-1}\begin{bmatrix}B_t \\ \bar{c}_x^t\end{bmatrix}(x_t^+ - x_t^\star) + R,
\end{aligned} \quad (34)$$

noting that the $\bar{x}_t$ terms cancel in simplifying line 3 of (34).

Since $(K_t)^{-1}$, B_t and $\bar{c}_x^t$ are uniformly bounded by Assumptions 1, it follows after rearranging (34) that

$$\begin{aligned}
u_t^+ - u_t^\star &= O\left(\max(\|x_t^+ - x_t^\star\|, \|\mathbf{w}^\star - \bar{\mathbf{w}}\|^2)\right) \\
\phi_t^+ - \phi_t^\star &= O\left(\max(\|x_t^+ - x_t^\star\|, \|\mathbf{w}^\star - \bar{\mathbf{w}}\|^2)\right).
\end{aligned} \quad (35)$$

It remains to show that $x_t^+ - x_t^\star = R$ for all $t \in [N]$, which we do using an inductive argument. The base case of $t = 1$ holds since $x_1^\star = x_1^+ = \hat{x}_1$. Now suppose $x_t^+ - x_t^\star = R$ for some $t \in [N-1]$. Then by (35), it follows that $w_t^+ - w_t^\star = R$, thus $w_t^+ - \bar{w}_t = w_t^\star - \bar{w}_t + R$ and so $w_t^+ - \bar{w}_t = O(\|\mathbf{w}^\star - \bar{\mathbf{w}}\|)$.

In addition to (25), Taylor's theorem applied to f yields

$$x_{t+1}^+ = \bar{x}_{t+1} + \bar{f}_x^t(x_t^+ - \bar{x}_t) + \bar{f}_u^t(u_t^+ - \bar{u}_t) + R, \quad (36)$$

from which we subtract (25) and use $w_t^+ - w_t^\star = R$ to yield $x_{t+1}^+ - x_{t+1}^\star = R$. Additionally, it follows from (35) that $u_{t+1}^+ - u_{t+1}^\star = R$ and $\phi_{t+1}^+ - \phi_{t+1}^\star = R$, and so $w_{t+1}^+ -$

$w^\star_{t+1} = R$. We conclude that $\mathbf{w}^+ - \mathbf{w}^\star = R$, and by setting M as the constant in R, we have shown that (32) holds.

Finally, to show (33), we select $\epsilon < 1/M$ and substitute $\|\mathbf{w}^\star - \bar{\mathbf{w}}\| < \epsilon$ into (32) for the desired result. ∎

Remark 2: Replacing λ^t with V^t invalidates Prop. 1, since (23) evaluated at $\bar{\mathbf{w}}$ is in general, inconsistent with (14a).

VI. Extension for Inequality Constraints

In this section, we present an extension of Alg. 1 for solving inequality constrained problems of form

$$\begin{aligned} \underset{\mathbf{x},\mathbf{u}}{\text{minimise}} \quad & J(x_{1:N}, u_{1:N}) \coloneqq \textstyle\sum_{t=1}^{N} \ell(x_t, u_t) \\ \text{subject to} \quad & x_1 = \hat{x}_1, \\ & x_{t+1} = f(x_t, u_t) \quad \text{for} \quad t \in [N-1], \\ & c(x_t, u_t) = 0, \quad u \geq 0, \quad \text{for } t \in [N]. \end{aligned} \tag{37}$$

A simple way to extend Alg. 1 is to use a (primal) barrier interior point method as in [28], which approximately solves a sequence of equality constrained barrier sub-problems, indexed by barrier parameters $\{\mu_j\}$, with the property that $\lim_{j\to\infty} \mu_j = 0$. Each sub-problem is given by (1), after replacing running cost ℓ with the *barrier cost*

$$\varphi_{\mu_j}(x_t, u_t) \coloneqq \ell(x_t, u_t) - \mu \sum_{i=1}^{m} \ln(u_t^{(i)}). \tag{38}$$

Instead, our proposed extension adopts a primal-dual interior point framework, similar to [31] and [27]. Concretely, we introduce dual variables for the inequality constraints denoted by $\mathbf{z} \coloneqq z_{1:N}$. The barrier sub-problems described above are then replaced with the *perturbed KKT conditions* given by (3), $z_t^\star \odot u_t^\star - \mu_j e = 0$ and $z_t^\star \geq 0$. Primal-dual methods are in general, numerically better conditioned than their primal counterparts e.g., see [24, Ch. 19] and [27], [31].

A. Changes to the Backward and Forward Passes

The primal-dual extension of FilterDDP requires minor changes to various equations in Sec. IV, which are described in this section. First, the termination criteria (7) for sub-problem j is replaced by $E_{\mu_j}(\mathbf{w}, \boldsymbol{\lambda}) < \kappa_\epsilon \mu_j$, where

$$E_{\mu_j}(\mathbf{w}, \boldsymbol{\lambda}) \coloneqq \max\left\{ E(\mathbf{w}, \boldsymbol{\lambda}), \max_t \|z_t \odot u_t - \mu_j e\| \right\}, \tag{39}$$

and $\kappa_\epsilon > 0$ is a parameter. Simply put, the perturbed complementarity slackness conditions are added.

If (39) is satisfied for the current iterate, then the sub-problem is considered solved and μ_j is updated using

$$\mu_{j+1} = \max\left\{ \frac{\epsilon_{\text{tol}}}{10}, \min\left\{ \kappa_\mu \mu_j, \mu_j^{\theta_\mu} \right\} \right\}, \tag{40}$$

where $\kappa_\mu \in (0,1)$ and $\theta_\mu \in (1,2)$. This formula was proposed by [34] and used in IPOPT [27], and encourages a superlinear decrease of μ_j. The algorithm is deemed to have converged if $E_0(\mathbf{w}_k, \boldsymbol{\lambda}_k) < \epsilon_{\text{tol}}$.

a) Backward Pass: We replace (12) with

$$\begin{bmatrix} H_t + \Sigma_t + \delta_w I & A_t \\ A_t^\top & -\delta_c I \end{bmatrix} \begin{bmatrix} \alpha_t & \beta_t \\ \psi_t & \omega_t \end{bmatrix} = - \begin{bmatrix} (\hat{Q}_u^t)^\top & B_t \\ \bar{c}^t & \bar{c}_x^t \end{bmatrix}, \tag{41}$$

where $\bar{U}_t \coloneqq \operatorname{diag}(\bar{u}_t)$, $\bar{Z}_t \coloneqq \operatorname{diag}(\bar{z}_t)$, $\Sigma_t \coloneqq \bar{U}_t^{-1}\bar{Z}_t$ and $\hat{Q}_u^t \coloneqq \bar{Q}_u^t - \mu e^\top \bar{U}_t^{-1}$.

b) Forward Pass: In addition to (15), we apply the update rule for dual variables z_t given by

$$z_t^+(x_t^+, \gamma) = \bar{z}_t + \gamma \chi_t + \zeta_t(x_t^+ - \bar{x}_t), \tag{42}$$

where $\chi_t = \mu \bar{U}_t^{-1} e - \bar{z}_t - \Sigma_t \alpha_t$ and $\zeta_t = -\Sigma_t \beta_t$. Also, a fraction-to-the-boundary condition [27] given by

$$u_t^+(x_t^+, \gamma) \geq (1-\tau)\bar{u}_t, \quad z_t^+(x_t^+, \gamma) \geq (1-\tau)\bar{z}_t \quad \forall t, \tag{43}$$

where $\tau = \max\{\tau_{\min}, 1-\mu\}$ for some $\tau_{\min} > 0$ must be satisfied for a step size γ to be accepted as the next iterate.

Finally, for filter condition (16), we replace ℓ with φ_{μ_j} for sub-problem j and replace $\bar{Q}_u^t$ with $\hat{Q}_u^t$ in (17b).

c) Barrier sub-problem update step: The final modification to Alg. 1 is that Step 3. of Alg. 1 is replaced with:

3. *Check convergence of overall problem.* If $E_0(\mathbf{w}_k, \boldsymbol{\lambda}_k) < \epsilon_{\text{tol}}$ then STOP (converged). If $k = \text{max\_iters}$ then STOP (not converged).

3.1. *Check convergence of barrier problem.* If $E_{\mu_j}(\mathbf{w}_k, \boldsymbol{\lambda}_k) < \kappa_\epsilon \mu_j$ then:

3.1.1. Compute μ_{j+1} and τ_{j+1} from (40) and (43) and set $j \leftarrow j+1$.

3.1.2. Re-initialise the filter $\mathcal{F}_k \coloneqq \left\{(\theta, \mathcal{L}) \in \mathbb{R}^2 : \theta \geq \theta_{\max}\right\}$.

3.1.3. If $k = 0$, repeat step 3.1., otherwise continue at 4.

Finally, Alg. 1 requires additional parameters $\tau_{\min} > 0$, $\mu_{\text{init}} > 0$, $\kappa_\epsilon > 0$, $\kappa_\mu \in (0,1)$ and $\theta_\mu \in (1,2)$.

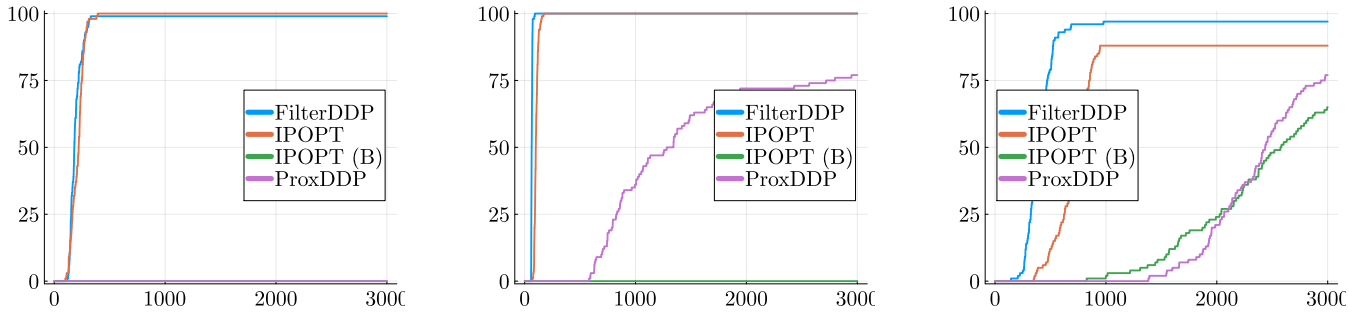
VII. Numerical Simulations

We evaluate FilterDDP on three planning tasks with nonlinear equality constraints: 1) a contact-implicit cartpole swing-up task with Columb friction, 2) a contact-implicit acrobot swing-up task with hard joint limits and, 3) a contact-implicit non-prehensile manipulation task.

A. Comparison Methods

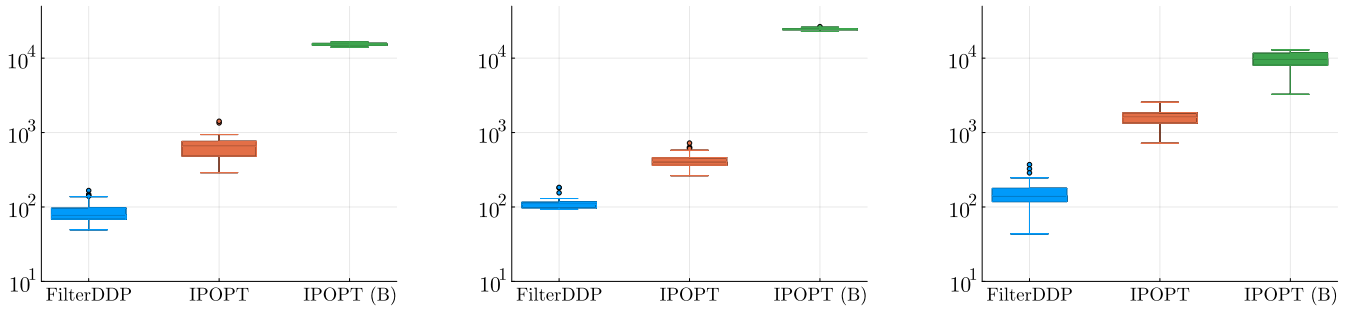
FilterDDP is compared against IPOPT [27] with full second-order derivatives and an L-BFGS variant which we refer to as IPOPT (B), as well the AL-DDP algorithm ProxDDP [22]. FilterDDP uses the Julia Symbolics.jl package [35], ProxDDP uses CasADi [36] and IPOPT uses the JuMP.jl package [37] for computing function derivatives. For the acrobot and manipulation tasks, we implement FilterDDP using statically-sized arrays and manual implementations of linear algebra operations[3] (avoiding external libraries such as LAPACK [38]), which is more efficient for small numbers of states, control variables and constraints thanks to compiler optimisations. For cartpole however, we instead implement FilterDDP using LAPACK for numerical linear algebra, which is more efficient for larger problem sizes. In our experiments, we report total wall clock time for FilterDDP and IPOPT. For ProxDDP, we do not report timing results since we used the (slower) Python interface.

[3]https://github.com/JuliaArrays/StaticArrays.jl



(a) Acrobot Contact (b) Cartpole Friction (c) Manipulation

Fig. 1: Results for the three planning tasks. For a)-c), the x-axis represents iteration count and the y-axis is the number of OCPs which converged to the error tolerance of 10^{-7} for Filter DDP and IPOPT and 10^{-5} for ProxDDP and IPOPT (B).



(a) Acrobot Contact (b) Cartpole Friction (c) Manipulation

Fig. 2: Total wall clock (ms) for each method over each OCP class. FilterDDP attains a mean wall clock time of 14.5% and 0.6% of IPOPT and IPOPT (B), respectively for the acrobot problem, a mean wall clock time of 27.1% and 4.5% of IPOPT and IPOPT (B), respectively for the cartpole problem and finally a mean wall clock time of 10.3% and 1.7% of IPOPT and IPOPT (B), respectively for the manipulation problem.

B. Experimental Setup

The solver parameters selected for FilterDDP across all experiments are identical to their IPOPT counterparts described in [27], except we reduce the superlinear barrier update term to $\theta_\mu = 1.2$ and increase the default barrier parameter to $\mu_{\text{init}} = 1$. FilterDDP are limited to 1,000 iterations while ProxDDP and IPOPT (B) are limited to 3,000 iterations. Optimality error tolerances are set to 10^{-7} for IPOPT and FilterDDP and 10^{-5} for ProxDDP and IPOPT (B). For ProxDDP, we applied a grid search per planning task to select the initial AL parameter μ_0 to minimise constraint violation, while default parameters were used for FilterDDP, IPOPT and IPOPT (B). All specific parameter values are provided in the Julia code. For each task, we create 100 individual OCPs by varying the dynamics parameters (e.g., link lengths and masses for acrobot) and actuation limits. All experiments were limited to one CPU core and performed on a Lenovo ThinkPad T14s Gen 5 with an Intel® Core™ Ultra 7 155U and 32GB of RAM, running Ubuntu 24.04.3 LTS and Julia version 1.13.0-beta3 compiled using the -O3 optimisation level.

C. Cartpole Swing-Up with Coulomb Friction

The first task is the cartpole swing-up task with Coulomb friction applying to the cart and pendulum, introduced by [19]. Frictional forces obey the principle of maximum dissipation, and the resultant OCP is derived by a variational time-stepping framework [17], [20]. Frictional forces are resolved jointly with the rigid-body dynamics by introducing constraints.

The states $x \in \mathbb{R}^4$ include the prior and current configurations of the system, while $u \in \mathbb{R}^{15}$ includes the cart actuation force, the next configuration of the system, frictional forces and finally, dual and slack variables relating to the contact constraints [17]. For each OCP, we vary the mass of the cart and pole, length of the pole and friction coefficients. See [19] and the supplementary for a detailed description of the OCP.

D. Acrobot Swing-Up with Joint Limits

This task is an acrobot swing-up task, where limits at $+\pi/2$ and $-\pi/2$ are imposed on the elbow joint, introduced by [19]. The joint limits are enforced by an impulse, which is only applied when a joint limit is reached. Intuitively, the acrobot is allowed to "slam" into a joint limit, and the impulse takes on the appropriate value to ensure the joint limits are not violated. Similar to cartpole, a variational time-stepping framework [17], [19] is applied to encode the contact dynamics.

The states $x \in \mathbb{R}^4$ include the prior and current configurations of the system, while $u \in \mathbb{R}^7$ includes the elbow torque, the next configuration, impulses at the joint limits and slack variables for the complementarity constraints [17].

For each OCP, we vary the link lengths. See [19] and the supplementary for a description of the OCP. An example trajectory of impulses at the joint limits is given in Fig. 3.

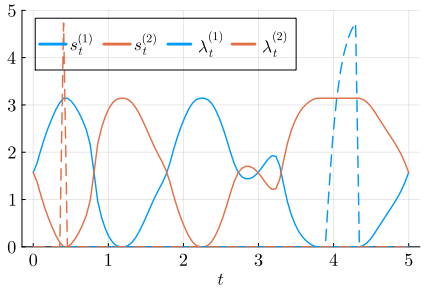


Fig. 3: Acrobot example. s_t is the signed distance to the $\pi/2$ and $-\pi/2$ joint limits and λ_t denotes the impulses. Around the 4s mark, the acrobot is "leaning into" the joint limit.

E. Non-Prehensile Manipulation

Our final task is a non-prehensile manipulation planning task, described in detail in [10], where a manipulator (the pusher) is tasked with using its end-effector to push a rectangular block (the slider) from a start to a goal configuration, while avoiding an obstacle. The dynamics of the pusher-slider problem includes multiple sticking and sliding contact modes, which are encoded through complementarity constraints.

The states $x \in \mathbb{R}^4$ include the pose of the slider and pusher in a global coordinate frame. Controls $u \in \mathbb{R}^6$ includes contact forces between the pusher and the slider, relative pusher/slider velocity and slack variables for the obstacle avoidance and complementarity constraints. We also apply inequality constraints to bound the contact force magnitudes and pusher/slider velocity and include an obstacle avoidance constraint. The objective function is a sum of a quadratic penalty on contact force and a high weight linear penalty on the slack variables (a so-called *exact* penalty [17]). For each OCP, we vary the coefficient of friction, obstacle size and location. Fig. 4 illustrates sample trajectories.

F. Summary Discussion of Results

We present the success rate per planning task across all 100 OCPs as a function of iteration count in Fig. 1. In addition, we present timing results in Fig. 2. FilterDDP is significantly faster compared to both IPOPT and IPOPT (B), yielding average wall clock times of 10-27% of IPOPT and 1-5% of IPOPT (B) across the three planning tasks. Furthermore, FilterDDP attains a comparable level of robustness to IPOPT in either similar or much fewer iterations.

Notably, the evaluated first-order methods ProxDDP and IPOPT (B) fails to solve the acrobot example entirely, with ProxDDP yielding feasible but degenerate solutions despite extensive parameter tuning, and IPOPT (B) yielding infeasible solutions. Furthermore, the iteration count for the remaining tasks for both ProxDDP and IPOPT (B) is significantly higher even with a relaxed termination tolerance, with neither method reliably converging to a local optima within a reasonable iteration count. This finding highlights the importance of using a robust second-order solver as opposed to a first-order for solving OCPs with challenging constraints.

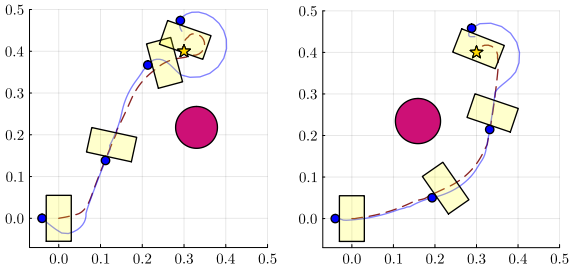
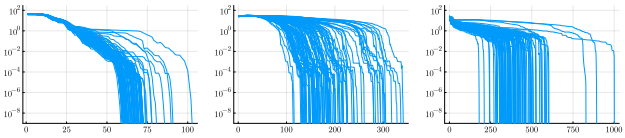


Fig. 4: Pusher and slider trajectories for two instances of the non-prehensile manipulation problem. The pusher and slider trajectories are marked by the blue and brown lines.



(a) Cartpole Friction (b) Acrobot Contact (c) Manipulation

Fig. 5: Local quadratic convergence of FilterDDP. The x-axis measures iteration count and the y-axis measures $\|\bar{u}_{1:N} - u^\star_{1:N}\|_2$, where $u^\star_{1:N}$ is the optimal point found by FilterDDP.

In Tab. I, we report the results of ablating the two critical design choices for the FilterDDP algorithm, 1) replacing the cost with the Lagrangian for step acceptance in (16), (20) and, 2) replacing Hessians (11) with perturbed Hessians (13). Finally, Fig. 5 plots the convergence of FilterDDP for all OCPs across all planning tasks. Local quadratic convergence is observed in practice, validating the formal result in Sec. V.

Remark 3: We experimented with a *Gauss-Newton* approximation to H_t (e.g., as in [15]), however, it is significantly less reliable than using full second-order derivatives, analogous to the performance gap between IPOPT and IPOPT (B).

	$\mathcal{L} \to J$	$\lambda_t \to V_x^t$	$\mathcal{L}, \lambda_t \to J, V_x^t$
Cartpole (I)	$66 \to 66$	$66 \to 76$	$66 \to 77$
Acrobot (I)	$181 \to 179$	$181 \to 209$	$181 \to 265$
Manip. (F)	$1 \to 5$	$1 \to 78$	$1 \to 63$

TABLE I: Ablation of algorithm design. (I) and (F) indicates that iteration count and no. of failures, resp. are reported.

VIII. Conclusion and Future Work

In this paper, we have presented a line-search filter differential dynamic programming algorithm for equality constrained optimal control problems. Important design choices for the iterates and filter criterion are proposed and validated both analytically and empirically on contact-implicit trajectory planning problems arising in robotics. Furthermore, a rigorous proof of local quadratic convergence is provided, generalising prior results on unconstrained DDP [39]. Future work will investigate applying FilterDDP to high-dimensional locomotion problems in legged robots, by integration of efficient rigid-body dynamics libraries [40] and demonstrating

FilterDDP in a control policy on hardware. Furthermore, formally establishing the global convergence of FilterDDP is another promising direction for future work.

REFERENCES

[1] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Science Robotics*, 2021.

[2] J. Y. Goh, T. Goel, and J. Christian Gerdes, "Toward Automated Vehicle Control Beyond the Stability Limits: Drifting Along a General Path," *Journal of Dynamic Systems, Measurement, and Control*, 2019.

[3] P. M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, and A. D. Prete, "Optimization-based control for dynamic legged robots," *IEEE Transactions on Robotics*, 2024.

[4] F. Farshidian, E. Jelavic, A. Satapathy, M. Giftthaler, and J. Buchli, "Real-time motion planning of legged robots: A model predictive control approach," in *IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids)*, 2017.

[5] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive locomotion through nonlinear model-predictive control," *IEEE Transactions on Robotics*, 2023.

[6] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *American Control Conference (ACC)*, 2021.

[7] X. Zhang, A. Liniger, and F. Borrelli, "Optimization-based collision avoidance," *IEEE Transactions on Control Systems Technology*, 2021.

[8] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science Robotics*, 2023.

[9] T. Xue, H. Girgin, T. S. Lembono, and S. Calinon, "Demonstration-guided optimal control for long-term non-prehensile planar manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[10] J. Moura, T. Stouraitis, and S. Vijayakumar, "Non-prehensile planar manipulation via trajectory optimization with complementarity constraints," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[11] W. Yang and M. Posa, "Dynamic On-Palm Manipulation via Controlled Sliding," in *Proceedings of Robotics: Science and Systems*, 2024.

[12] F. R. Hogan and A. Rodriguez, "Reactive planar non-prehensile manipulation with hybrid model predictive control," *The International Journal of Robotics Research*, 2020.

[13] D. Q. Mayne and D. H. Jacobson, *Differential Dynamic Programming*. Springer Series in Operations Research and Financial Engineering, American Elsevier Publishing Company, 1970.

[14] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, "Feedback MPC for Torque-Controlled Legged Robots," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.

[15] C. Mastalli, S. P. Chhatoi, T. Corbéres, S. Tonneau, and S. Vijayakumar, "Inverse-Dynamics MPC via Nullspace Resolution," *IEEE Transactions on Robotics*, 2023.

[16] S. Katayama and T. Ohtsuka, "Efficient solution method based on inverse dynamics for optimal control problems of rigid body systems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[17] Z. Manchester, N. Doshi, R. J. Wood, and S. Kuindersma, "Contact-implicit trajectory optimization using variational integrators," *The International Journal of Robotics Research*, 2019.

[18] M. Posa, C. Cantu, and R. Tedrake, "A direct method for trajectory optimization of rigid bodies through contact," *The International Journal of Robotics Research*, 2014.

[19] T. A. Howell, S. Le Cleac'h, S. Singh, P. Florence, Z. Manchester, and V. Sindhwani, "Trajectory Optimization with Optimization-Based Dynamics," *IEEE Robotics and Automation Letters*, 2022.

[20] S. Le Cleac'h, T. A. Howell, S. Yang, C.-Y. Lee, J. Zhang, A. Bishop, M. Schwager, and Z. Manchester, "Fast Contact-Implicit Model Predictive Control," *IEEE Transactions on Robotics*, 2024.

[21] S. E. Kazdadi, J. Carpentier, and J. Ponce, "Equality Constrained Differential Dynamic Programming," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[22] W. Jallet, A. Bambade, E. Arlaud, S. El-Kazdadi, N. Mansard, and J. Carpentier, "ProxDDP: Proximal Constrained Trajectory Optimization," *IEEE Transactions on Robotics*, 2025.

[23] T. A. Howell, B. E. Jackson, and Z. Manchester, "ALTRO: A Fast Solver for Constrained Trajectory Optimization," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.

[24] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering, 2nd ed., 2006.

[25] A. Wächter and L. T. Biegler, "Line Search Filter Methods for Nonlinear Programming: Motivation and Global Convergence," *SIAM Journal on Optimization*, 2005.

[26] A. Wächter and L. T. Biegler, "Line Search Filter Methods for Nonlinear Programming: Local Convergence," *SIAM Journal on Optimization*, vol. 16, no. 1, pp. 32–48, 2005.

[27] A. Wächter and L. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, 2006.

[28] S. Prabhu, S. Rangarajan, and M. Kothare, "Differential dynamic programming with stagewise equality and inequality constraints using interior point method," 2024.

[29] L.-z. Liao and C. A. Shoemaker, "Advantages of differential dynamic programming over newton's method for discrete-time optimal control problems," tech. rep., Cornell University, 1992.

[30] R. Bellman and R. Kalaba, *Dynamic Programming and Modern Control Theory*. Academic Press, Elsevier Science, 1965.

[31] A. Pavlov, I. Shames, and C. Manzie, "Interior Point Differential Dynamic Programming," *IEEE Transactions on Control Systems Technology*, 2021.

[32] J. R. Bunch and L. Kaufman, "Some stable methods for calculating inertia and solving symmetric linear systems," *Mathematics of Computation*, 1977.

[33] G. Poole and L. Neal, "The rook's pivoting strategy," *Journal of Computational and Applied Mathematics*, 2000. Numerical Analysis 2000. Vol. III: Linear Algebra.

[34] R. H. Byrd, G. Liu, and J. Nocedal, "On the local behavior of an interior point method for nonlinear programming," in *Numerical Analysis 1997* (D. F. Griffiths and D. J. Higham, eds.), pp. 37–56, Reading, MA, USA: Addison–Wesley Longman, 1997.

[35] S. Gowda, Y. Ma, A. Cheli, M. Gwozdz, V. B. Shah, A. Edelman, and C. Rackauckas, "High-performance symbolic-numerics via multiple dispatch," *arXiv preprint arXiv:2105.03949*, 2021.

[36] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 2019.

[37] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, 2023.

[38] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, third ed., 1999.

[39] L.-Z. Liao and C. Shoemaker, "Convergence in unconstrained discrete-time differential dynamic programming," *IEEE Transactions on Automatic Control*, 1991.

[40] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiraux, O. Stasse, and N. Mansard, "The Pinocchio C++ library : A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *IEEE/SICE International Symposium on System Integration (SII)*, 2019.